National University of Singapore
School of Computing
CS1101S: Programming Methodology
Semester I, 2026/2027

S11
Searching and Sorting

Problems:

1. Recall the `binary_search` function from Lecture L10A, which only tells you whether a value is present in a sorted list:

```
def binary_search(A, v):
    low = 0
    high = len(A) - 1
    while low <= high:
        mid = math_floor((low + high) / 2)
        if v == A[mid]:
            break
        elif v < A[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return low <= high
```

Using the same halving idea, write a function `insertion_point` that takes a sorted list `A` and a value `v`, and returns the index at which `v` should be inserted into `A` to keep it sorted. If `v` already occurs in `A`, return the index of its leftmost occurrence.

```
def insertion_point(A, v):
    # ???
```

Example calls:

```
insertion_point([1, 3, 5, 7, 9], 5)  # returns 2 (5 is already at index 2)
insertion_point([1, 3, 5, 7, 9], 6)  # returns 3 (between 5 and 7)
insertion_point([1, 3, 5, 7, 9], 0)  # returns 0 (before everything)
insertion_point([1, 3, 5, 7, 9], 10) # returns 5 (after everything)
```

What is the order of growth (in time) of `insertion_point`?

2. Recall `selection_sort` and `insertion_sort` from Lecture L10A. Both sort a list of length `n` of arbitrary comparable values.

   For each of the two algorithms, give the order of growth (in terms of the number of comparisons made) in the *best* case and in the *worst* case, and describe an input that achieves each case. (Comparisons means evaluations of `A[i] < A[j]` or similar – not swaps.)